

Smart Greenhouse Backend

REST ingestion, auditable device commands, and MQTT delivery

Node.js 22

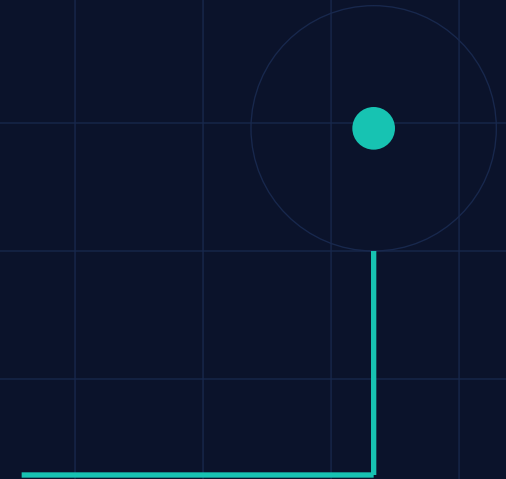
TypeScript

Fastify

PostgreSQL

MQTT

Rival • Backend Engineer Assignment • Project walkthrough



The product in one minute

01

Capture

Accept temperature and humidity readings through a strict JSON API.

02

Control

Translate ON/OFF requests into device-specific MQTT commands.

03

Trace

Persist sensor events and command delivery status for auditability.

Designed for

Client applications

send readings and control requests



Backend service

validates, persists, and publishes



IoT devices

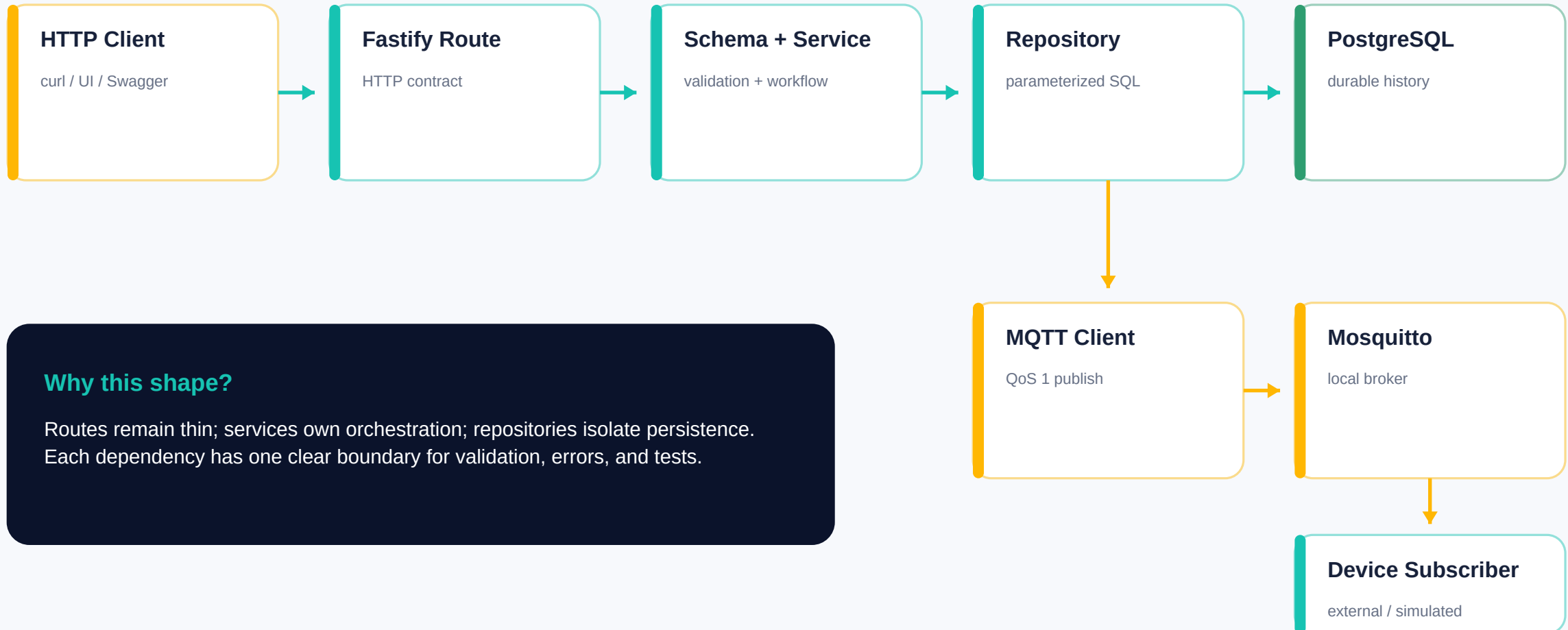
subscribe to their command topic

Requirements mapped to working code

REQUIREMENT	IMPLEMENTATION	EVIDENCE
Store sensor data	POST /sensor-data → service → parameterized INSERT	201 + DB assertion
Control devices	Persist PENDING → MQTT QoS 1 → PUBLISHED / FAILED	Exact topic tested
Validate inputs	TypeBox schema + AJV + SQL CHECK constraints	400 scenarios
Operable locally	Multi-stage image + Compose health-gated services	3-container stack
Expose health	API + live PostgreSQL query + MQTT connection state	200 / 503

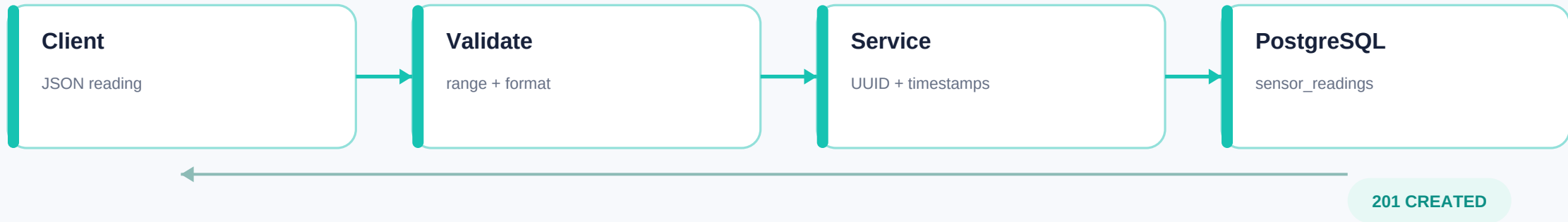
Scope noteassignment.md and jobdesc.md are not present in the repository; this mapping uses README.md and verified behavior.

Layered architecture keeps responsibilities explicit



Two workflows, one auditable backend

A. SENSOR INGESTION — SYNCHRONOUS HTTP



B. DEVICE COMMAND — DURABLE INTENT BEFORE PUBLISH



Small API surface, strict contracts

METHOD	ENDPOINT	PURPOSE	VALIDATION / CHECK	RESULT
POST	/sensor-data	Persist a reading	device ID, -50...100 °C, 0...100%, ISO date	201
POST	/device-control	Issue ON / OFF	safe device ID; exact enum	200 / 503
GET	/status	Dependency health	live DB query + MQTT state	200 / 503
GET	/docs	Interactive API docs	OpenAPI generated from schemas	200
Consistent failure envelope		{ "success": false, "error": { "code": "VALIDATION_ERROR", "message": "..." } }		

The database stores events and command lifecycle

sensor_readings

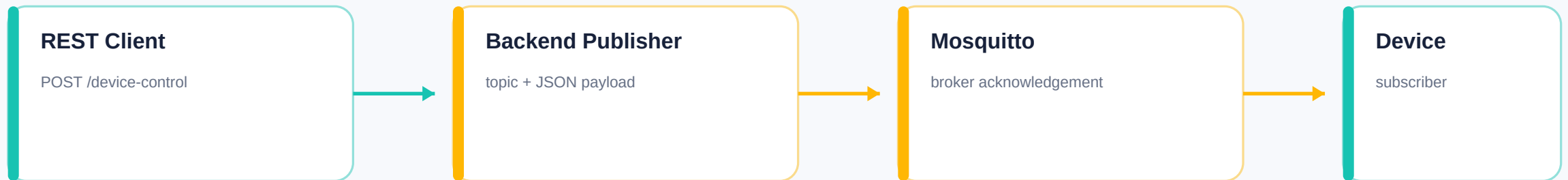
PK	id	UUID
	device_id	varchar(128)
	temperature	double precision
	humidity	double precision
	recorded_at	timestampz
	created_at	timestampz

device_commands

PK	id	UUID
	request_id / device_id	varchar
	command	ON OFF
	mqtt_topic	varchar(255)
	status	PENDING PUBLISHED FAILED
	error_message	text, nullable
	issued_at / published_at	timestampz

No foreign key by design: device_id is an external identifier; readings are append-only events. Composite indexes optimize per-device time queries.

MQTT decouples command producers from devices



Topic `greenhouse/control/{device_id}`

QoS 1 • retain false • clean session

Reconnect

2-second reconnect interval

Timeout

5-second connect timeout

Safety

device ID blocks / + # injection

Real-time delivery exists on MQTT—not the client API

Implemented

- Backend publishes device commands immediately
- Subscribers receive broker-forwarded messages
- Health endpoint exposes current MQTT connection state

Not implemented

- No WebSocket or SSE connection to frontend clients
- No inbound sensor subscription through MQTT
- No device-level execution acknowledgement

Interpretation

PUBLISHED means the MQTT publish callback succeeded—not that a physical actuator completed the command. Device acknowledgements would require an additional status topic and state transition.

Defense in depth around a compact service

INPUT

- TypeBox + AJV contracts
- No type coercion
- No extra fields
- 1 MiB body limit

ABUSE

- 100 requests/minute/IP
- MQTT topic injection blocked
- Parameterized SQL
- Safe client errors

RESILIENCE

- DB pool timeouts
- MQTT reconnect
- Health dependencies
- Graceful shutdown

Production gap

No API authentication/authorization; local Mosquitto allows anonymous clients. Credentials are supported by env vars, but broker ACL/TLS is not configured.

Verification covers the complete integration path

13 / 13

integration tests passed

0

npm audit vulnerabilities

PASS

strict TypeScript check

PASS

production build

01**Happy paths**

health, sensor persistence, MQTT publish + DB status

02**Contract failures**

missing fields, wrong types, ranges, malformed JSON

03**Protection**

1 MiB limit, topic injection, global rate limit, 404

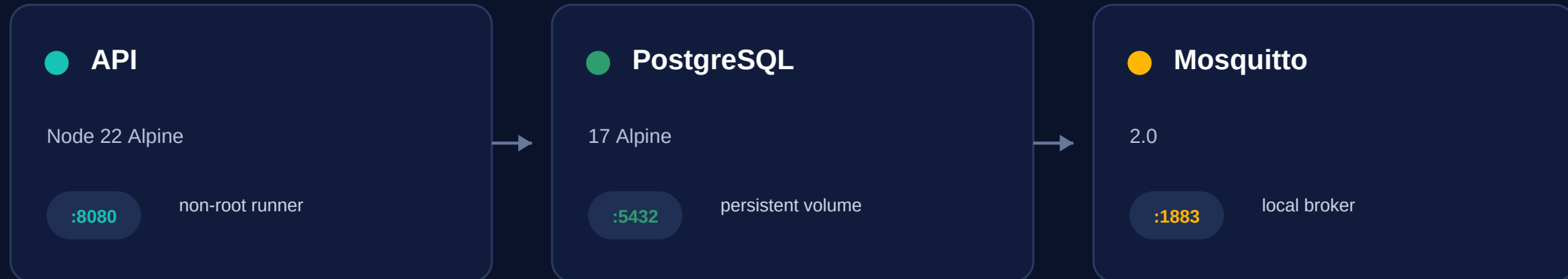
04**Event semantics**

repeated readings persist as separate events

Executed on 20 Sep 2026 against the running Docker stack.

One command starts an isolated local environment

Docker Compose



Startup order is health-gated: PostgreSQL + Mosquitto → API. Migration 001_init.sql runs automatically before listening.

Engineering decisions expose clear trade-offs

AUDIT BEFORE DELIVERY

Persist PENDING first

Commands remain traceable when MQTT is unavailable.

Next: transactional outbox

TOPIC SAFETY

Constrain device_id

Blocks wildcard and path injection before topic construction.

Defense at API boundary

FAILURE CLARITY

Typed dependency errors

Clients get stable codes; internal details stay in logs.

Avoids leaking internals

LOCAL REPEATABILITY

Compose + health checks

A complete test environment without physical hardware.

Fast interviewer demo

A focused backend with honest production boundaries

01

Reliable ingestion

Validated sensor events persist with timestamps and database constraints.

02

Auditable control

Every device command records intent and MQTT delivery outcome.

03

Verified delivery

The full HTTP → database → MQTT path is integration-tested.

Next production steps

Auth + broker ACL/TLS • device acknowledgements • outbox/retry • observability

Thank you

Questions?